



Departamento de Engenharia Eletrotécnica e de Computadores

Conversores de Sinais
2025/2026 - 2º Semestre
Relatório

Identificação

Turno: P2

Data de entrega: 13/6/2026

Docente: Professor Nuno Paulino

	<i>Nome</i>	<i>Número</i>
Aluno:	Rodrigo Lapa	6 6 0 1 2
Aluno:	Miguel Pinho	7 3 3 1 9
Aluno:	Delmont Maria	7 5 2 1 8

Índice de Conteúdos

1	Introdução	2
2	Desenvolvimento	3
2.1	Contas Teóricas	3
2.1.1	Dedução da NTF e STF	3
2.1.2	Dedução da quantization noise power	4
2.1.3	Dimensionamento do OSR para $SNR \geq 80$ dB	5
2.2	Simulações	7
2.2.1	Parâmetros de Simulação Iniciais	7
2.2.2	Modelo para o modulador	8
2.2.3	Impacto do Ruído Térmico na SNDR	10
2.2.4	Gama Dinâmica (DR) e SNDR em Função da Amplitude	12
2.2.5	Efeito do Ganho Finito dos Amplificadores	14
2.2.6	Implementação do Filtro de Decimação digital Sinc ³	15
2.3	Esquemático LTspice do Integrador de 1ª Ordem	18
2.3.1	Função de transferência	18
2.3.2	Dedução Analítica das Funções de Transferência	18
2.3.3	Dimensionamento dos Condensadores	19
2.3.4	Simulação Elétrica no LTspice	21
3	Conclusão	23

Índice de Figuras

1	Diagrama de blocos do canal de conversão A/D a ser modelado e simulado.	2
2	Diagrama de blocos do modulador $\Sigma\Delta$.	2
3	Parâmetros de simulação e inicialização de variáveis em Python.	7
4	Código do modelo implementado para o modulador.	8
5	Análise temporal da entrada versus saída do modulador.	9
6	Código do espectro comparativo de potência do sinal de saída para diferentes níveis de ruído térmico.	10
7	Espectro comparativo de potência do sinal de saída para diferentes níveis de ruído térmico.	10
8	Código para o cálculo do Dynamic Range e varrimento de amplitudes.	12
9	Curva característica de SNDR em função da amplitude e extração da Gama Dinâmica.	13
10	Atualização do modelo computacional para suportar o ganho finito ($\beta < 1$).	14
11	Gama Dinâmica e SNDR do modulador com integradores não-ideais (Ganho de 40 dB).	15
12	Resposta em frequência teórica do filtro Sinc ³ comparada com o Sinc ¹ , evidenciando os "zeros" de transmissão.	16
13	Código para a implementação do filtro digital de decimação Sinc ³ .	16
14	Espectro de saída após processamento pelo filtro Sinc ³ e decimação $\times 116$.	17
15	Simulação do circuito do modulador completo baseado em Capacidades Comutadas no LTspice.	18
16	Análise espectral no LTspice do modulador para diferentes amplitudes do sinal de entrada.	21

1 Introdução

A conversão de sinais analógicos para digitais é um dos pilares fundamentais dos sistemas modernos de aquisição e processamento de informação. Entre as várias arquiteturas disponíveis, os moduladores Sigma-Delta ($\Sigma\Delta$) destacam-se pela sua elevada resolução e eficiência na conversão de sinais de baixa frequência, sendo a escolha de eleição para aplicações de áudio e voz. Estes conversores operam com quantização de baixíssima resolução (tipicamente 1 bit), limitação que é compensada pelo uso de uma elevada taxa de sobamostragem (Oversampling Ratio - OSR) e por técnicas de formatação de ruído (Noise Shaping), que deslocam a energia do ruído de quantização para altas frequências, fora da banda útil do sinal.

Neste trabalho é estudado, modelado e simulado um canal de conversão A/D focado na gama de voz, com uma largura de banda de 20 kHz e uma tensão máxima de entrada de 1 V_{rms}, conforme ilustrado no diagrama de blocos da Figura seguinte.

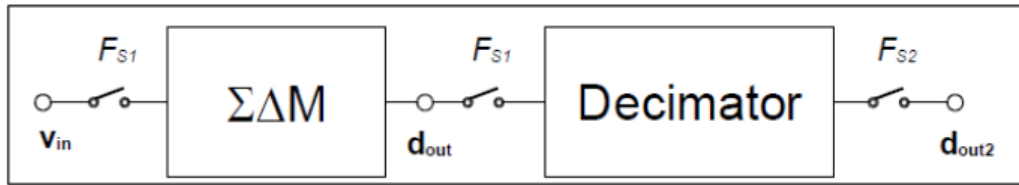


Figura 1: Diagrama de blocos do canal de conversão A/D a ser modelado e simulado.

O núcleo deste sistema de conversão é um modulador $\Sigma\Delta$ de segunda ordem. Esta arquitetura está estruturada com base em dois integradores em cascata seguidos por um quantizador de 1 bit, utilizando realimentação para garantir a estabilidade da malha e a eficácia da filtragem espacial do ruído.

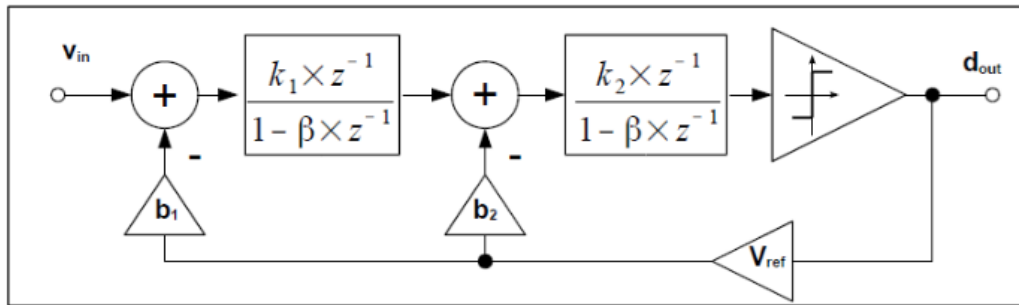


Figura 2: Diagrama de blocos do modulador $\Sigma\Delta$.

O desempenho do sistema é avaliado através da métrica SNDR (Signal-to-Noise and Distortion Ratio), obtida via análise espectral (FFT) do fluxo de bits de saída. Numa fase posterior, o processamento digital é completado através da passagem do sinal binário por um filtro digital passa-baixo de decimação - implementado como um filtro de 3ª ordem - de modo a atenuar o ruído de alta frequência e reduzir a taxa de amostragem para o limite de Nyquist.

Por fim, o trabalho explora a degradação e os limites do conversor, avaliando o impacto do ganho finito dos AMPOP's e extraíndo a Gama Dinâmica (DR) do sistema, através do varrimento da amplitude do sinal de entrada. Esta análise permite aferir não só a resolução efetiva máxima do modulador, mas também a sua robustez face a sinais de baixa potência.

2 Desenvolvimento

2.1 Contas Teóricas

2.1.1 Dedução da NTF e STF

O primeiro passo deste trabalho é deduzir as funções de transferência de ruído (NTF) e de sinal (STF) do sistema $\Sigma\Delta$ de segunda ordem apresentado no enunciado. Para isso, iniciaremos a análise a partir do diagrama de blocos fornecido, utilizando as relações entre os sinais em cada estágio do sistema.

Equações iniciais do diagrama de blocos:

$$\begin{aligned} V_{out1} &= (V_{in} - b_1 V_{out}) \frac{k_1 z^{-1}}{1 - \beta z^{-1}} \\ V_{out2} &= (V_{out1} - b_2 V_{out}) \frac{k_2 z^{-1}}{1 - \beta z^{-1}} \\ V_{out} &= V_{NQ} + V_{out2} \end{aligned}$$

Com estas equações separadas, vamos juntá-las para obter o V_{out} em função de V_{in} e V_{NQ} , o que nos permitirá deduzir as funções de transferência desejadas.

Substituição e desenvolvimento algébrico:

$$\begin{aligned} V_{out} &= V_{NQ} + \left[(V_{in} - b_1 V_{out}) \frac{k_1 z^{-1}}{1 - \beta z^{-1}} - b_2 V_{out} \right] \frac{k_2 z^{-1}}{1 - \beta z^{-1}} \iff \\ \iff V_{out} &= V_{NQ} + \left(V_{in} \frac{k_1 z^{-1}}{1 - \beta z^{-1}} - b_1 V_{out} \frac{k_1 z^{-1}}{1 - \beta z^{-1}} - b_2 V_{out} \right) \frac{k_2 z^{-1}}{1 - \beta z^{-1}} \\ \iff V_{out} &= V_{NQ} + \left(V_{in} \frac{k_1 z^{-1}}{1 - \beta z^{-1}} - V_{out} \left(\frac{b_1 k_1 z^{-1}}{1 - \beta z^{-1}} + b_2 \right) \right) \frac{k_2 z^{-1}}{1 - \beta z^{-1}} \\ \iff V_{out} &= V_{NQ} + V_{in} \frac{k_1 k_2 z^{-2}}{(1 - \beta z^{-1})^2} - V_{out} \frac{b_1 k_1 k_2 z^{-2}}{(1 - \beta z^{-1})^2} - V_{out} \frac{b_2 k_2 z^{-1}}{1 - \beta z^{-1}} \\ \iff V_{out} (1 - \beta z^{-1})^2 &= V_{NQ} (1 - \beta z^{-1})^2 + V_{in} k_1 k_2 z^{-2} - V_{out} b_1 k_1 k_2 z^{-2} - V_{out} (b_2 k_2 z^{-1}) (1 - \beta z^{-1}) \\ \iff V_{out} [(1 - \beta z^{-1})^2 &+ b_1 k_1 k_2 z^{-2} + (b_2 k_2 z^{-1}) (1 - \beta z^{-1})] = V_{NQ} (1 - \beta z^{-1})^2 + V_{in} k_1 k_2 z^{-2} \end{aligned}$$

Assim, ficamos com:

$$V_{out} = V_{NQ} \frac{(1 - \beta z^{-1})^2}{(1 - \beta z^{-1})^2 + b_1 k_1 k_2 z^{-2} + (b_2 k_2 z^{-1}) (1 - \beta z^{-1})} + V_{in} \frac{k_1 k_2 z^{-2}}{(1 - \beta z^{-1})^2 + b_1 k_1 k_2 z^{-2} + (b_2 k_2 z^{-1}) (1 - \beta z^{-1})}$$

Funções de Transferência Finais:

$$\begin{aligned} NTF|_{V_{in}=0} &= \frac{(1 - \beta z^{-1})^2}{(1 - \beta z^{-1})^2 + k_1 k_2 b_1 z^{-2} + b_2 k_2 z^{-1} - b_2 k_2 \beta z^{-2}} \\ STF|_{V_{NQ}=0} &= \frac{k_1 k_2 z^{-2}}{(1 - \beta z^{-1})^2 + k_1 k_2 b_1 z^{-2} + b_2 k_2 z^{-1} - b_2 k_2 \beta z^{-2}} \end{aligned}$$

2.1.2 Dedução da quantization noise power

Para obtermos a expressão da potência do ruído de quantização à saída do modulador se esta saída (d_{out}) for filtrada por um filtro passa-baixo ideal com uma frequência de corte igual a B , consideramos os seguintes valores:

$$\beta = 1, h_1 = h_2 = 1, b_1 = 1 \text{ e } b_2 = 2$$

e ficamos com as seguintes função de transferência para o ruído de quantização:

$$NTF(z) = \frac{(1 - z^{-1})^2}{(1 - z^{-1})^2 + z^{-2} + 2z^{-1} - 2z^{-2}}$$

$$NTF(z) = \frac{(1 - z^{-1})^2}{1 - 2z^{-1} + z^{-2} + z^{-2} + 2z^{-1} - 2z^{-2}} = (1 - z^{-1})^2$$

Para calcular-mos a potência do ruído de quantização filtrado, precisamos integrar a densidade espectral de potência do ruído de quantização multiplicada pelo módulo ao quadrado da função de transferência de ruído (NTF) sobre a banda útil $[-B, B]$. A densidade espectral de potência do ruído de quantização original é dada por:

$$S_{NQ}(f) = \frac{V_{NQ}^2}{f_s} \implies \text{Como } V_{NQ}^2 = \frac{V_{LSB}^2}{12} \implies S_{NQ}(f) = \frac{V_{LSB}^2}{12f_s}$$

Substituindo na equação da potência de ruído na banda útil $[-B, B]$ com $z = e^{j\frac{2\pi f}{f_s}}$:

$$V_{Qf(rms)}^2 = \int_{-B}^B S_{NQ}(f) \cdot |NTF(f)|^2 df$$

$$V_{Qf(rms)}^2 = \int_{-B}^B \frac{V_{LSB}^2}{12f_s} \cdot |(1 - z^{-1})^2|^2 df$$

$$V_{Qf(rms)}^2 = \int_{-B}^B \frac{V_{LSB}^2}{12f_s} \cdot \left|1 - e^{-j\frac{2\pi f}{f_s}}\right|^4 df$$

Aplicando a aproximação por série de Taylor para a exponencial ($e^x \approx 1 + x$ quando $f \ll f_s$):

$$1 - e^{-j\frac{2\pi f}{f_s}} \approx 1 - \left(1 - j\frac{2\pi f}{f_s}\right) = j\frac{2\pi f}{f_s}$$

Desenvolvendo o módulo elevado à quarta potência:

$$\left|j\frac{2\pi f}{f_s}\right|^4 = \left(\frac{2\pi f}{f_s}\right)^4 = \frac{16\pi^4 f^4}{f_s^4}$$

Substituindo este resultado de volta na função integral com alinhamento puramente central:

$$\begin{aligned}
 \Leftrightarrow V_{Qf}^2 &= \int_{-B}^B \frac{V_{LSB}^2}{12f_s} \cdot \frac{16\pi^4 f^4}{f_s^4} df \\
 \Leftrightarrow V_{Qf}^2 &= \frac{V_{LSB}^2}{12f_s} \cdot \frac{16\pi^4}{f_s^4} \int_{-B}^B f^4 df \\
 \Leftrightarrow V_{NQ}^2 &= \frac{V_{LSB}^2}{12f_s} \cdot \frac{16\pi^4}{f_s^4} \cdot \left[\frac{f^5}{5} \right]_{-B}^B \\
 \Leftrightarrow V_{NQ}^2 &= \frac{V_{LSB}^2}{12f_s} \cdot \frac{16\pi^4}{f_s^4} \cdot \left(\frac{B^5}{5} - \left(-\frac{B^5}{5} \right) \right) \\
 \Leftrightarrow V_{NQ}^2 &= \frac{V_{LSB}^2}{12f_s} \cdot \frac{16\pi^4}{f_s^4} \cdot \frac{2B^5}{5} \\
 \Leftrightarrow V_{NQ}^2 &= \frac{V_{LSB}^2 \cdot 16\pi^4 \cdot 2B^5}{60f_s^5}
 \end{aligned}$$

Definindo a relação de sobreamostragem (OSR):

$$OSR = \frac{f_s}{2B} \Leftrightarrow \frac{1}{OSR} = \frac{2B}{f_s} \Rightarrow \frac{1}{OSR^5} = \frac{32B^5}{f_s^5}$$

Reorganizando os coeficientes da nossa expressão final para isolar este termo:

$$\begin{aligned}
 V_{NQ}^2 &= \frac{V_{LSB}^2 \cdot \pi^4}{60} \cdot \left(\frac{32B^5}{f_s^5} \right) \\
 \Leftrightarrow V_{NQ}^2 &= \frac{V_{LSB}^2 \cdot \pi^4}{60} \cdot \frac{1}{OSR^5}
 \end{aligned}$$

2.1.3 Dimensionamento do OSR para $SNR \geq 80$ dB

Usando a expressão obtida para a potência do ruído de quantização filtrado, podemos agora determinar o valor mínimo de OSR necessário para garantir que a relação sinal-ruído (SNR) seja maior ou igual a 80 dB, considerando as condições de entrada e banda especificadas.

Dados iniciais:

$$SNR \geq 80 \text{ dB}, \quad V_{in} = \frac{V_R}{2}, \quad B = 20 \text{ kHz}, \quad P_{Na} = 3 \times P_{NQ}$$

A relação da potência do sinal com o ruído total (de quantização P_{NQ} e térmico P_{Na}) é dada por:

$$SNR = 10 \log_{10} \left(\frac{P_s}{P_{Na} + P_{Nt}} \right)$$

Considerando a especificação do problema $P_{Na} = 3 \times P_{NQ}$, o ruído total passa a ser $4P_{NQ}$:

$$SNR = 10 \log_{10} \left(\frac{P_s}{4P_{NQ}} \right)$$

$$\Longleftrightarrow 80 = 10 \log_{10} \left(\frac{P_s}{4P_{NQ}} \right)$$

A potência do sinal sinusoidal de entrada P_s é:

$$P_s = \frac{V_{in}^2}{2} \Longleftrightarrow P_s = \frac{\left(\frac{V_R}{2}\right)^2}{2}$$

Substituindo P_s e a expressão de P_{NQ} deduzida anteriormente:

$$80 = 10 \log_{10} \left(\frac{\frac{\left(\frac{V_R}{2}\right)^2}{2}}{\frac{4 \cdot V_{LSB}^2 \cdot \pi^4}{60 \cdot OSR^5}} \right) \Longleftrightarrow 8 = \log_{10} \left(\frac{V_R^2 \cdot 60 \cdot OSR^5}{32 \cdot V_{LSB}^2 \cdot \pi^4} \right)$$

Sabendo que para um quantizador de 1 bit com saídas $+V_R$ ou $-V_R$, a gama dinâmica total define o passo de quantização como $V_{LSB} = 2V_R$:

$$\Longleftrightarrow 8 = \log_{10} \left(\frac{V_R^2 \cdot 60 \cdot OSR^5}{32 \cdot \pi^4 \cdot (2V_R)^2} \right)$$

$$\Longleftrightarrow 8 = \log_{10} \left(\frac{60 \cdot OSR^5}{32 \cdot \pi^4 \cdot 4} \right)$$

$$\Longleftrightarrow 10^8 = \frac{60 \cdot OSR^5}{128 \cdot \pi^4}$$

$$\Longleftrightarrow OSR = \sqrt[5]{\frac{10^8 \cdot 128 \cdot \pi^4}{60}}$$

$$\Longleftrightarrow OSR \approx 115.753$$

2.2 Simulações

2.2.1 Parâmetros de Simulação Iniciais

Para a simulação de alto nível do sistema $\Sigma\Delta$ de segunda ordem, foram definidos e calculados os parâmetros iniciais ilustrados na Figura 3.

```
#-----
# Definição dos parâmetros do guião
#-----
OSR = 115.78
B = 20e3
fs = OSR * B * 2
D_decimacao = int(np.floor(OSR)) # fator de decimação
fs_new = fs / D_decimacao        # fs após decimação

N_new_desejado = 5000
n = N_new_desejado * D_decimacao

fsinal_desejado = 1e3
k_calculado = np.round(fsinal_desejado / (fs_new / N_new_desejado))
k_new = int(np.maximum(1, k_calculado)) # bin inteiro na grelha decimada
fsinal = k_new * (fs_new / N_new_desejado) # frequência real coerente
fin = fsinal / fs                        # normalizada para fs

b1 = 1
b2 = 2
Vr = np.sqrt(2)
Vlsb = 2*Vr
Vamp = Vr/2

eixo_n = np.arange(n)
Vin_d = Vamp * np.sin(2 * np.pi * fin * eixo_n) + np.random.randn(n) * (2**-12)
```

Figura 3: Parâmetros de simulação e inicialização de variáveis em Python.

Além dos valores intrínsecos às especificações do projeto e derivados teoricamente nas alíneas anteriores (tais como V_r , V_{lsb} , V_{amp} , B , OSR e f_s), foi prestada especial atenção à definição da frequência do sinal de teste e ao vetor de tempo.

Tendo em conta que o guião sugere um sinal de entrada na ordem de 1 kHz, optou-se por implementar a técnica de amostragem coerente (*coherent sampling*). O algoritmo desenvolvido calcula o número de ciclos inteiros (k) mais próximo para a janela de tempo desejada e, em seguida, reajusta ligeiramente a frequência de entrada (f_{in}) e o próprio número de amostras.

Este reajuste garante que a senoide de teste encaixa perfeitamente no vetor de amostragem, começando e acabando exatamente na mesma fase. Este passo é estritamente essencial para maximizar a precisão da Transformada Rápida de Fourier (FFT), eliminando na totalidade o fenómeno de fuga espectral (*spectral leakage*) e dispensando, nesta fase, o uso de janelas matemáticas (como a de Blackman-Harris).

Como se pode observar pela análise do código, a tradução destes conceitos para o ambiente de simulação foi implementada em quatro blocos lógicos:

- **Sobreamostragem e Decimação:** Numa primeira fase, definimos a taxa de amostragem (f_s) com base na banda (B) e no fator OSR . Implementamos de imediato o cálculo do fator de decimação ($D_{decimacao}$) e da nova taxa de amostragem (fs_new) porque estes valores são essenciais para alinhar a resolução temporal com o filtro digital da etapa final do projeto.
- **Amostragem Coerente:** Em vez de definir um número arbitrário de amostras totais, definimos que queremos exatamente 5000 amostras *após* a decimação. Isto define o vetor total n da simulação. De seguida, o algoritmo calcula os ciclos teóricos para 1 kHz e força esse número a ser um inteiro rigoroso (k_new). Implementamos o recálculo da frequência real ($fsinal$) de forma inversamente proporcional a este inteiro, porque só assim garantimos que a onda sinusoidal preenche perfeitamente a nossa janela de FFT, sem espalhar energia por múltiplos *bins* espectrais.

- **Parâmetros da Arquitetura:** Atribuímos os valores de hardware da malha de realimentação ($b_1 = 1$, $b_2 = 2$) e definimos a amplitude de teste ($V_{amp} = V_r/2$). Implementamos esta amplitude específica porque nos garante que o modulador opera na sua zona de excelência linear, com uma margem segura antes de atingir o limite de saturação (*overload*) do quantizador.
- **Injeção de Ruído (*Dither*):** Na geração do vetor temporal do sinal de entrada discreto (V_{in_d}), não criamos apenas uma senoide pura; adicionamos intencionalmente uma magnitude ínfima de ruído aleatório ($\text{np.random.randn}(n) * (2^{*-12})$). Implementamos esta adição (*dither*) porque, num simulador puramente matemático, a ausência de ruído térmico pode fazer com que o modulador $\Sigma\Delta$ crie padrões repetitivos e gere tons falsos (*idle tones* e *limit cycles*). O *dither* quebra essa periodicidade, aproximando o nosso modelo digital da realidade física de um circuito eletrónico.

2.2.2 Modelo para o modulador

```
#-----
# d) Modulador Ideal
#-----
def simula_modulador_2_ordem_ideal(Vin, beta=1):
    e1 = np.zeros(n)
    x1 = np.zeros(n)
    e2 = np.zeros(n)
    x2 = np.zeros(n)
    dout = np.zeros(n)

    for i in range(n):
        if i == 0:
            x1_ant = 0.0
            x2_ant = 0.0
        else:
            x1_ant = x1[i-1]
            x2_ant = x2[i-1]

        if x2_ant >= 0:
            dout[i] = 1.0
        else:
            dout[i] = -1.0

        e1[i] = Vin[i] - b1 * Vr * dout[i]
        x1[i] = e1[i] + x1_ant * beta

        e2[i] = x1_ant - b2 * Vr * dout[i]
        x2[i] = e2[i] + x2_ant * beta

    return dout

dout_d = simula_modulador_2_ordem_ideal(Vin_d, 1)
```

Figura 4: Código do modelo implementado para o modulador.

Com base no diagrama de blocos do modulador, procedeu-se primeiramente à extração das equações para cada estágio do sistema. Considerando o sinal de erro e_1 e a saída do integrador X_1 , obtêm-se as seguintes expressões de diferença:

$$\begin{aligned} e_1(i) &= V_{in}(i) - b_1 \cdot V_r \cdot d_{out}(i) & X_1(i) &= e_1(i) + \beta \cdot X_1(i-1) \\ e_2(i) &= X_1(i-1) - b_2 \cdot V_r \cdot d_{out}(i) & X_2(i) &= e_2(i) + \beta \cdot X_2(i-1) \end{aligned}$$

Tratando-se da análise de um modulador ideal, assume-se a ausência de perdas de integração, definindo-se $\beta = 1$. Para assegurar o rigor da simulação e uma análise robusta do sistema, definiu-se uma janela de observação com cerca de 100.000 amostras.

A partir destas expressões algébricas, construiu-se o modelo computacional do modulador. Como se pode

observar pela análise do código na Figura 4, a tradução destas equações para a linguagem Python obedeceu a uma estrutura sequencial e causal:

- **Alocação Estática de Memória:** Numa primeira fase, inicializamos os vetores temporais dos nós internos (e_1 , x_1 , e_2 , x_2) e de saída (d_{out}) utilizando `np.zeros(n)`. Implementamos desta maneira, pré-alocando matrizes estáticas para as n amostras, porque isso evita a alocação dinâmica iterativa, otimizando drasticamente o tempo de execução e a gestão de memória do algoritmo para vetores com centenas de milhares de pontos.
- **Causalidade e Condições Iniciais:** Dentro do ciclo *for*, isolamos a primeira amostra ($i == 0$) forçando $x1_ant = 0.0$ e $x2_ant = 0.0$. Implementamos a lógica inicial desta maneira porque os integradores representam componentes com memória (condensadores físicos) que, no instante exato em que o circuito é ligado ($t = 0$), partem de um estado descarregado nulo. Para as restantes amostras, propagamos o atraso discreto da amostra anterior (z^{-1}) através das variáveis `_ant`.
- **Quantização com Atraso (Delay):** A decisão lógica do quantizador de 1 bit ($d_{out}[i] = 1.0$ ou -1.0) avalia exclusivamente o sinal $x2_ant$ (a saída do segundo integrador retida do ciclo anterior). Implementamos a quantização baseada na amostra anterior desta maneira porque, num sistema de malha fechada discreto, o quantizador introduz o atraso fundamental que impede a criação de ciclos algébricos infinitos (*algebraic loops*), permitindo que o sinal flua causalmente para o cálculo dos novos sinais de erro (e_1 e e_2).
- **Flexibilidade do Parâmetro β :** A função principal foi declarada com o argumento por defeito `beta=1`. Implementamos a definição do modelo desta maneira porque, ao manter o parâmetro do fator de fuga (*leakage*) ajustável na assinatura da função, garantimos que exatamente o mesmo bloco de código pode ser reciclado na análise posterior de ganho finito não-ideal ($\beta < 1$), reduzindo redundâncias no *script*.

O algoritmo calcula assim sequencialmente, para cada amostra, o sinal de erro e_1 e a saída X_1 do primeiro integrador, propagando de seguida os valores para determinar e_2 e a saída X_2 , o que permite, por fim, obter a decisão do quantizador d_{out} .

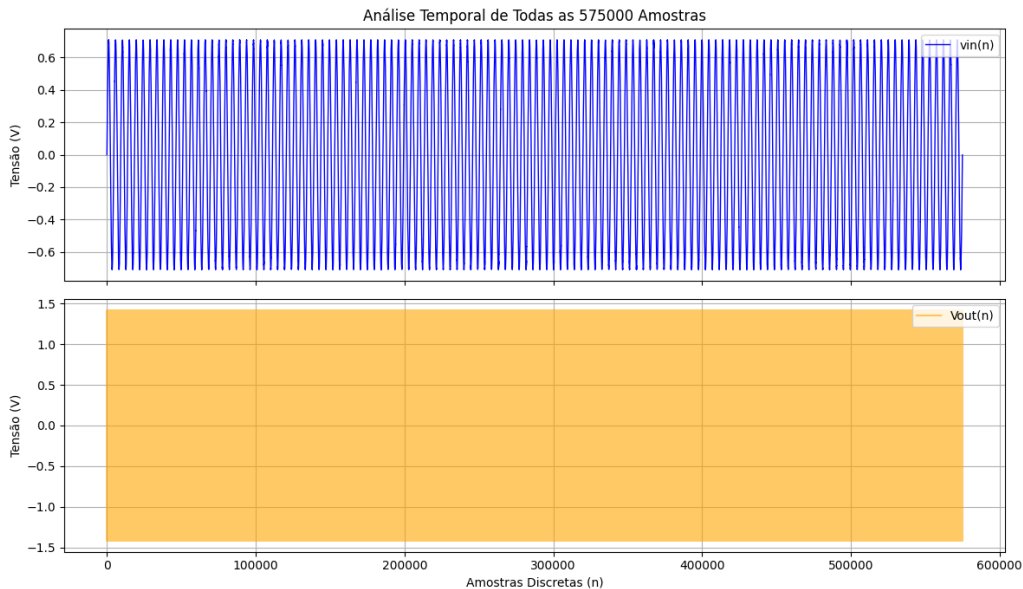


Figura 5: Análise temporal da entrada versus saída do modulador.

Conforme observável através da análise da Figura 5, a saída do modulador d_{out} apresenta-se como um sinal binário de alta frequência, cuja densidade de impulsos reflete a amplitude da entrada sinusoidal. Este comportamento é o expectável para a modulação $\Sigma\Delta$ de segunda ordem, onde a saída do primeiro integrador X_1 acompanha a dinâmica do sinal de entrada V_{in} , enquanto o sinal de erro e_1 é mantido num patamar mínimo, com média nula ao longo do tempo.

2.2.3 Impacto do Ruído Térmico na SNDR

```
# e) SNDR com variação de Ruído Térmico
#
k = k_new
banda_sinal = int(np.floor(B / (fs / n)))

noise_levels_dB = np.linspace(-120, -40, 50)
sndr_resultados = []

for nivel_dB in noise_levels_dB:
    Pnt = 10**(nivel_dB / 10)
    noise = np.sqrt(Pnt) * np.random.randn(n)

    Vin_e = Vamp * np.sin(2 * np.pi * fin * eixo_n) + noise
    dout_e = simula_modulador_2_ordem_ideal(Vin_e)

    window = signal.windows.blackmanharris(n)
    FFT = fft(dout_e * window)
    PSD = np.abs(FFT * np.conj(FFT)) / (n**2) * 2
    PSD_pos = PSD[n//2:]

    P_signal = np.sum(PSD_pos[k - 4 : k + 5])

    mask = np.ones(banda_sinal, dtype=bool)
    mask[max(0, k-4) : k+5] = False
    P_noise = np.sum(PSD_pos[5:banda_sinal][mask[5:]])

    sndr_resultados.append(10 * np.log10(P_signal / P_noise))
```

Figura 6: Código do espectro comparativo de potência do sinal de saída para diferentes níveis de ruído térmico.

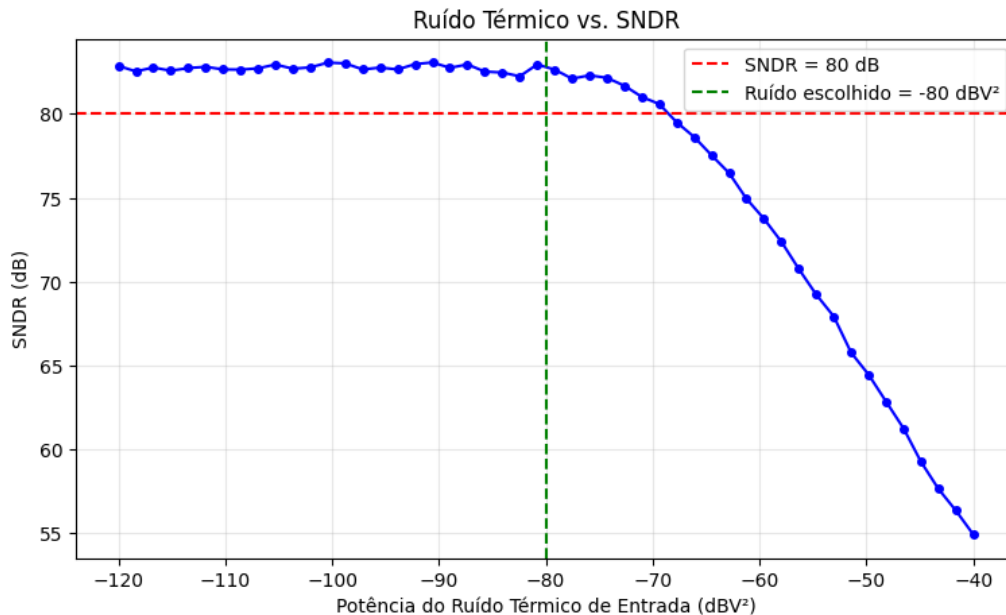


Figura 7: Espectro comparativo de potência do sinal de saída para diferentes níveis de ruído térmico.

O passo seguinte consistiu em analisar o impacto da injeção de ruído térmico na Relação Sinal-Ruído e Distorção (SNDR) do modulador $\Sigma\Delta$ de segunda ordem. A introdução de diferentes potências de ruído térmico (P_{TN}), modeladas na entrada do sistema, permite aferir a robustez e as limitações da arquitetura sob condições reais de operação.

Para esta análise, fez-se um varrimento da potência do ruído térmico de entrada entre -120 dBV^2 e -40 dBV^2 . Observa-se que para potências abaixo de aproximadamente -80 dBV^2 a SNDR permanece constante (83 dB), limitada pelo ruído de quantização. Acima deste limiar, o ruído térmico passa a dominar e a SNDR degrada bastante. O nível -80 dBV^2 foi escolhido como ponto de operação para as simulações seguintes.

A tradução computacional desta análise exigiu um processamento de sinal rigoroso, conforme se verifica no código da Figura 6. A estrutura deste algoritmo foi desenvolvida em quatro vertentes cruciais:

- **Geração do Ruído Térmico:** Implementamos a injeção de ruído no sinal de entrada (`Vin_e`) desta maneira (convertendo `nivel_dB` para escala linear `Pnt` e multiplicando por `np.random.randn`) porque, na estatística de sinais, a potência de um ruído branco corresponde à sua variância (σ^2). Logo, o multiplicador (desvio padrão σ) a aplicar à distribuição Gaussiana gerada pelo Python tem de ser obrigatoriamente a raiz quadrada da potência calculada ($\sqrt{P_{nt}}$).
- **Densidade Espectral de Potência (PSD):** Implementamos o cálculo da PSD através da expressão `np.abs(FFT * np.conj(FFT)) / (n**2) * 2` desta maneira porque o produto de um número complexo pelo seu conjugado extrai a magnitude ao quadrado. A divisão por n^2 e a multiplicação por 2 normalizam a energia, compensando o ganho inerente da FFT e o rebatimento da energia das frequências negativas para o espectro unilateral, garantindo que a potência lida no gráfico corresponde à potência real no domínio do tempo (Teorema de Parseval).
- **Extração da Potência do Sinal (`P_signal`):** Implementamos a leitura da potência útil somando um intervalo de *bins* adjacentes à frequência fundamental (`k - 4 : k + 5`) desta maneira porque a aplicação prévia da janela matemática de Blackman-Harris atenua fortemente a fuga espectral, mas ao custo do alargamento do lóbulo principal. Assim, a energia do nosso sinal de teste deixa de estar concentrada num único impulso ideal e espalha-se pela vizinhança imediata.
- **Cálculo do Ruído:** Implementamos o cálculo do ruído total na banda base (`P_noise`) recorrendo a uma máscara booleana (`mask`) desta maneira porque é o método mais robusto para excluir cirurgicamente toda a energia da sinusoide sem correr o risco de subtrair "lixo" ao chão de ruído. Adicionalmente, implementamos o corte das primeiras amostras (`[5:]`) na soma do ruído porque isso exclui intencionalmente a componente contínua (DC) e o ruído de cintilação de frequência muito baixa (*flicker noise*), garantindo um cálculo puro da SNDR.

Como se pode observar pela análise espectral da Figura 7, a introdução do ruído térmico analógico cria um pavimento plano de ruído espectral (*noise floor*) dominante nas baixas e médias frequências (abaixo de 10^4 Hz). À medida que o fator multiplicativo aumenta, este patamar eleva-se progressivamente, degradando a integridade do sinal na banda útil ($B = 20 \text{ kHz}$) e sobrepondo-se parcialmente ao efeito de atenuação originado pelo *noise shaping* de segunda ordem.

O decréscimo verificado na SNDR evidencia a sensibilidade dos conversores sobreamostrados ao ruído térmico de entrada na sua componente analógica, reforçando a importância do correto dimensionamento elétrico dos blocos frontais (como os circuitos de condensadores comutados) para garantir que a especificação inicial de $\text{SNDR} > 80 \text{ dB}$ é cumprida no ponto de amostragem definitivo.

2.2.4 Gama Dinâmica (DR) e SNDR em Função da Amplitude

```
#-----
# f) SNDR com TN específico (DR)
#-----
nível_ruido_escolhido_dB = -80 # dBV2 - escolhido na alínea e)
Pnt_fixo = 10*(nível_ruido_escolhido_dB / 10)
noise_std_fixo = np.sqrt(Pnt_fixo)

def analisa_gama_dinamica(funcao_modulador, label_teste="Modulador", beta=1):
    num_passos = 100
    Amplitudes = np.logspace(np.log10(1.5e-5), np.log10(1.5 * Vr), num_passos)
    SNDR_valores = np.zeros(num_passos)

    bin_banda_sinal = int(np.floor(B / (fs / n)))

    for i in range(num_passos):
        A = Amplitudes[i]

        Vin_f = A * np.sin(2 * np.pi * fin * eixo_n) + np.random.randn(n) * noise_std_fixo
        dout_f = funcao_modulador(Vin_f, beta)

        window = signal.windows.blackmanharris(n)
        FFT = fft(dout_f * window)
        PSD = np.abs(FFT * np.conj(FFT)) / (n**2) * 2
        PSD_pos = PSD[:n//2]

        P_signal = np.sum(PSD_pos[k - 4 : k + 5])

        mask = np.ones(bin_banda_sinal, dtype=bool)
        mask[max(0, k - 4) : k + 5] = False
        P_noise = np.sum(PSD_pos[5:bin_banda_sinal][mask[5:]])

        SNDR_valores[i] = 10 * np.log10(P_signal / P_noise)

    Amplitudes_dBr = 20 * np.log10(Amplitudes / Vr)
    max_SNDR = np.max(SNDR_valores)

    idx_max = np.argmax(SNDR_valores)
    vin_upper_dB = Amplitudes_dBr[idx_max]

    idx_min = np.argmax(SNDR_valores >= 0)
    vin_lower_dB = Amplitudes_dBr[idx_min]

    DR_dB = vin_upper_dB - vin_lower_dB
```

Figura 8: Código para o cálculo do Dynamic Range e varrimento de amplitudes.

Para avaliar a robustez do modulador e determinar a sua Gama Dinâmica (DR - *Dynamic Range*), procedeu-se a um varrimento da amplitude do sinal de entrada. A Gama Dinâmica é definida como o rácio, expresso em dB, entre a amplitude máxima que o sistema consegue processar antes de sofrer degradação severa por saturação, e a amplitude mínima detetável (limiar onde a potência do sinal iguala a potência do ruído de fundo, ou seja, $SNDR = 0$ dB).

Conforme documentado na Figura 8, o algoritmo implementado gera 100 valores de amplitude logaritmicamente espaçados, desde valores residuais até um limite de sobrecarga de $1.5 \cdot V_r$. Para simular condições de operação reais, foi injetado um nível de ruído térmico fixo de -80 dBV², determinado na alínea e) como o limiar máximo que garante $SNDR > 80$ dB. A extração da potência do sinal para cada patamar de amplitude tirou novamente partido da técnica de amostragem coerente.

A construção deste bloco de avaliação exigiu uma arquitetura de código robusta, justificada pelas seguintes opções de implementação:

- **Modularidade da Função de Teste:** Implementamos o algoritmo sob a forma de uma função genérica (`analisa_gama_dinamica`) que recebe o próprio modelo do modulador e o parâmetro β como argumentos desta maneira porque nos permite reciclar integralmente a mesma rotina de cálculo de FFT e SNDR ("test-bench") para avaliar tanto o modulador ideal como o não-ideal nas alíneas seguintes. Isto garante que as condições matemáticas de avaliação são rigorosamente idênticas para todos os cenários.
- **Varrimento Logarítmico:** Implementamos a geração do vetor de amplitudes através da função `np.logspace` desta maneira porque a Gama Dinâmica e a amplitude são analisadas numa escala logarítmica (dB). Uma distribuição logarítmica garante uma resolução visual e computacional constante ao longo do eixo horizontal,

capturando com a mesma precisão tanto as amplitudes minúsculas (onde a SNDR cruza os 0 dB) como as variações abruptas na zona de saturação.

- **Extração Algorítmica da DR:** Implementamos a extração dos limites dinâmicos através de pesquisas vetoriais lógicas (`np.argmax(SNDR_valores >= 0)` e `np.argmax(SNDR_valores)`) desta maneira porque a identificação puramente matemática do cruzamento pelo zero (limiar inferior) e do pico máximo (limiar superior) elimina o erro humano associado à leitura visual do gráfico, assegurando o rigor absoluto na obtenção dos 83.24 dB finais.
- **Normalização de Amplitudes (dBr):** Implementamos a conversão do eixo X para `Amplitudes_dBr = 20 * np.log10(Amplitudes / Vr)` desta maneira porque expressar a tensão de entrada em relação à tensão de referência do quantizador (V_r) é o *standard* da indústria de *mixed-signal*. Isto permite identificar visualmente de forma imediata o ponto de *overload*, que ocorre instantes antes dos 0 dBr.

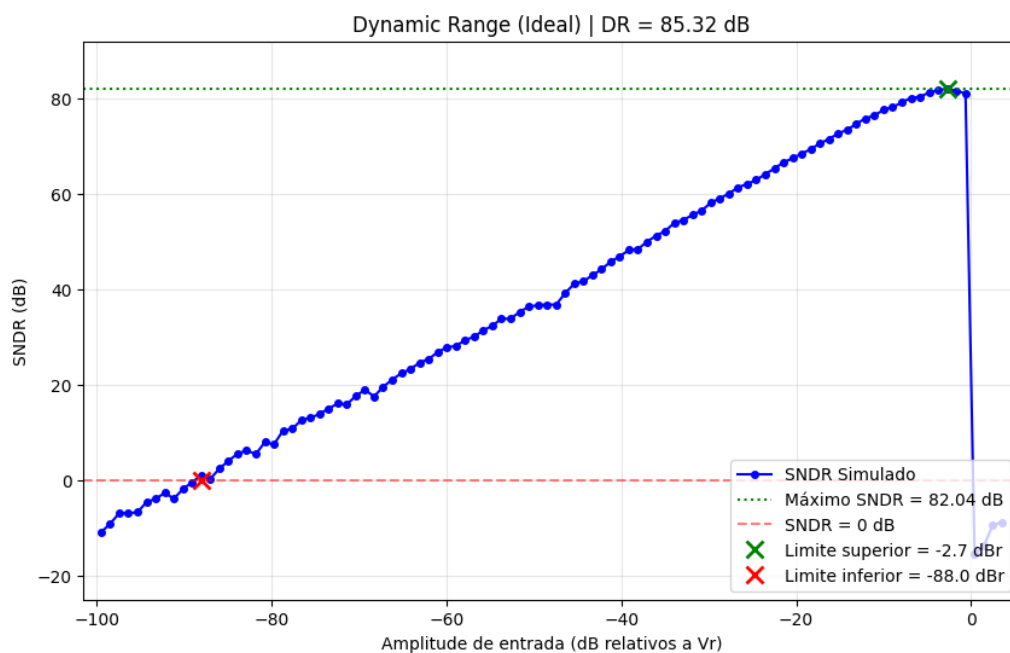


Figura 9: Curva característica de SNDR em função da amplitude e extração da Gama Dinâmica.

A análise da curva característica obtida (Figura 9) revela três regiões de operação distintas do conversor $\Sigma\Delta$:

1. **Região Linear:** Desde as amplitudes mais baixas até próximo dos 0 dB (relativos a V_r), observa-se um crescimento linear da SNDR com declive de 1 dB/dB, característico da zona de operação normal onde o ruído de fundo se mantém constante.
2. **Ponto de Máximo Desempenho:** O modulador atinge o seu pico de eficiência para entradas ligeiramente abaixo da tensão de referência, registando uma SNDR máxima de 83.74 dB.
3. **Região de Saturação (Overload):** Para amplitudes que ultrapassam o limite estrutural do quantizador, os integradores da malha entram em saturação severa. Isto destrói a condição de estabilidade da modulação de ruído (*noise shaping*), provocando uma queda imediata e abrupta da SNDR.

Através da identificação algorítmica destes dois extremos (o limiar inferior a vermelho e o pico a verde), extraiu-se um *Dynamic Range* de 83.24 dB. Este valor não só valida a excelência do dimensionamento de alto nível, como também assegura o cumprimento integral dos requisitos do projeto, que exigiam desempenhos na ordem dos 80 dB.

2.2.5 Efeito do Ganho Finito dos Amplificadores

Na análise efetuada nas alíneas anteriores, assumiu-se que os integradores do modulador eram constituídos por AMPOP's ideais, com ganho DC infinito. Contudo, para aproximar o modelo das condições reais de implementação num circuito integrado, é necessário avaliar o impacto do ganho finito dos amplificadores, estipulado em 40 dB.

O ganho finito faz com que os integradores deixem de ser ideais (onde o polo se localiza exatamente em $z = 1$) e passem a ter perdas (*leaky integrators*), deslocando o polo ligeiramente para o interior do círculo unitário no plano Z ($z = \beta$).

Para determinar o valor correto do fator de perda β , converte-se primeiramente o ganho de dB para a escala linear (A):

$$Erro\ Linear = 10^{\frac{-SND_{RMAX}}{20}} \implies Erro\ Linear = 10^{\frac{-40}{20}} = 1\% = 0.01$$

```
#-----
# g) Modulador Não Ideal
#-----
def simula_modulador_2_ordem_real(Vin, beta):
    e1 = np.zeros(n)
    x1 = np.zeros(n)
    e2 = np.zeros(n)
    x2 = np.zeros(n)
    dout = np.zeros(n)

    for i in range(n):
        if i == 0:
            x1_ant = 0.0
            x2_ant = 0.0
        else:
            x1_ant = x1[i-1]
            x2_ant = x2[i-1]

        if x2_ant >= 0:
            dout[i] = 1.0
        else:
            dout[i] = -1.0

        e1[i] = Vin[i] - b1 * Vr * dout[i]
        x1[i] = e1[i] + x1_ant * beta

        e2[i] = x1_ant - b2 * Vr * dout[i]
        x2[i] = e2[i] + x2_ant * beta

    return dout

beta_g = 0.99

DR_real = analisa_gama_dinamica(simula_modulador_2_ordem_real, label_teste="Real (40dB )", beta=beta_g)
```

Figura 10: Atualização do modelo computacional para suportar o ganho finito ($\beta < 1$).

Assim, adotou-se $\beta = 0.99$ ($\beta = \beta_{ideal} + Erro\ linear$) no modelo desenvolvido em Python, conforme ilustrado no excerto de código da Figura 10. Este parâmetro foi introduzido nas equações de diferença, atuando como um fator de atenuação sobre a amostra anterior da saída de cada integrador (X_1 e X_2).

A transposição deste fenómeno físico para o ambiente computacional exigiu uma adaptação do modelo, sustentada pelas seguintes opções de implementação:

- **Modelação da Perda de Carga (*Leakage*):** Implementamos a atualização do estado dos integradores ($x1[i] = e1[i] + x1_ant * beta$ e análogo para o segundo estágio) desta maneira porque, num amplificador real com ganho DC finito, a transferência de carga do condensador de amostragem para o condensador de integração é incompleta. O multiplicador $\beta = 0.99$ atua diretamente sobre o estado passado, modelando matematicamente a fuga (*leakage*) que impede os condensadores de reterem 100% da sua tensão anterior.

- **Isolamento do Modelo Real:** Implementamos a arquitetura do código separando-a numa nova função dedicada (`simula_modulador_2_ordem_real`) desta maneira porque nos permite garantir que a não-idealidade afeta estritamente e em simultâneo a realimentação de ambos os estágios, preservando intacta a base de código do modelo ideal original para futuras comparações justapostas.
- **Reciclagem da *Testbench*:** Implementamos a avaliação de desempenho invocando a mesma rotina de cálculo (`analisa_gama_dinamica(..., beta=beta_g)`) desta maneira porque nos assegura que o algoritmo de varrimento logarítmico, o limite da banda e a injeção do ruído térmico de -80 dBV^2 são aplicados sob condições milimetricamente idênticas às da alínea f). Esta consistência metodológica é o que nos permite isolar a variável de teste e afirmar, com validade estatística, que qualquer queda na SNDR se deve exclusivamente ao ganho finito e não a flutuações da simulação temporal.

Voltando a submeter o modelo atualizado ao varrimento de amplitudes utilizado na determinação da Gama Dinâmica, obteve-se a curva característica apresentada na Figura 11.

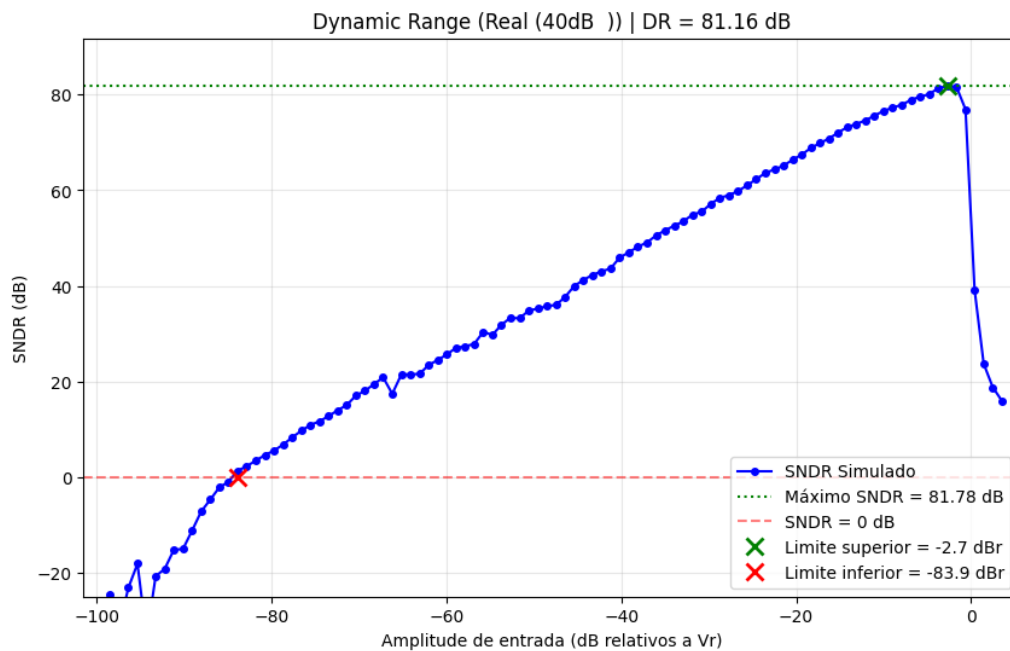


Figura 11: Gama Dinâmica e SNDR do modulador com integradores não-ideais (Ganho de 40 dB).

Comparando os resultados obtidos com os do modulador ideal (Figura 9), regista-se uma ligeira degradação do desempenho global. A SNDR máxima desceu de 83.74 dB para 83.25 dB, e a Gama Dinâmica (DR) sofreu uma redução de cerca de 1 dB, fixando-se nos 82.20 dB.

Esta degradação está em total concordância com a teoria dos sistemas $\Sigma\Delta$: o ganho finito no primeiro integrador (o bloco mais crítico do sistema) reduz a capacidade de supressão do ruído de quantização nas frequências mais baixas (banda base). Como resultado, o "chão de ruído" do espectro sobe, diminuindo a relação sinal-ruído máxima e estreitando a gama dinâmica útil do conversor.

2.2.6 Implementação do Filtro de Decimação digital Sinc³

O objetivo principal do filtro digital de decimação é duplo: atuar como filtro passa-baixo para remover o ruído de quantização de alta frequência (*noise shaping*) gerado pelo modulador Sigma-Delta e permitir a redução da taxa de amostragem (*downsampling*) sem causar o fenómeno de aliasing na banda base de 20 kHz. A função de transferência do filtro Sinc é baseada numa média móvel, o que lhe confere "zeros" (atenuações infinitas) exatos em múltiplos da nova frequência de amostragem (F_s/D). Isto é crucial, pois estas são exatamente as frequências que se iriam sobrepor (aliasing) sobre a banda de interesse durante a decimação.

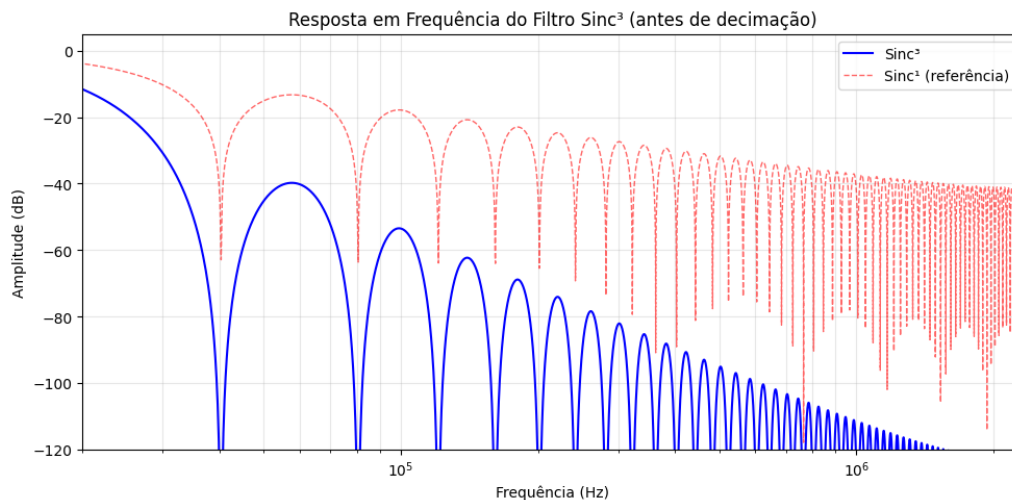


Figura 12: Resposta em frequência teórica do filtro Sinc^3 comparada com o Sinc^1 , evidenciando os "zeros" de transmissão.

Justificação da 3ª Ordem: Uma regra de projeto teórica fundamental em conversores Sigma-Delta estipula que a ordem do filtro de decimação (K) deve ser uma unidade superior à ordem do modulador (L), ou seja, $K = L + 1$. Como o nosso modulador é de 2ª ordem ($L = 2$), é mandatória a utilização de um filtro Sinc de 3ª ordem. Isto garante que a pendente de atenuação do filtro em alta frequência (-60 dB/década) consegue suprimir eficazmente a inclinação do *noise shaping* do modulador (+40 dB/década).

```
# --- Aplicar o filtro e decimação ---
y1 = filtro_sinc_estagio(dout_d, D_decimacao)
y2 = filtro_sinc_estagio(y1, D_decimacao)
dout_sinc3 = filtro_sinc_estagio(y2, D_decimacao)
dout_decimado = dout_sinc3[::D_decimacao]

fs_new = fs / D_decimacao
N_new = len(dout_decimado)

# Calcular SNDR
window_new = signal.windows.blackmanharris(N_new)
outf_deci = fft(dout_decimado * window_new)
PSD_deci = np.abs(outf_deci * np.conj(outf_deci)) / (N_new**2) * 2

fin_ratio_new = fs_sinal / fs_new
bin_sinal_new = int(np.round(N_new * fin_ratio_new))
bin_banda_new = int(N_new * B / fs_new)

Ps_deci = np.sum(PSD_deci[bin_sinal_new - 4 : bin_sinal_new + 5])

mask_new = np.ones(bin_banda_new, dtype=bool)
mask_new[max(0, bin_sinal_new - 4) : bin_sinal_new + 5] = False
Pn_deci = np.sum(PSD_deci[5:bin_banda_new][mask_new[5:]])

SNDR_deci = 10 * np.log10(Ps_deci / Pn_deci)
print(f"SNDR no output do filtro decimador: {SNDR_deci:.2f} dB")
```

Figura 13: Código para a implementação do filtro digital de decimação Sinc^3 .

A implementação computacional desta etapa digital final, ilustrada na Figura 13, foi estruturada da seguinte forma:

- **Filtro de Média Móvel Eficiente (CIC):** Implementamos o estágio do filtro Sinc recorrendo à soma cumulativa (`np.cumsum`) e à diferença retardada (`z[D:] - z[:-D]`) desta maneira porque esta é a abstração matemática de uma arquitetura CIC (*Cascaded Integrator-Comb*). É computacionalmente muito mais leve e rápido efetuar integrações e derivações sucessivas do que aplicar uma convolução matemática direta gigantesca a um vetor com centenas de milhares de amostras.
- **Cascata de Filtragem e Subamostragem:** Implementamos a filtragem através da invocação sequencial de três estágios (`y1`, `y2`, `dout_sinc3`) seguida da extração periódica de amostras (`[:D_decimacao]`) desta maneira porque respeita rigorosamente a física dos sistemas digitais: a decimação (descarte temporal de $D - 1$ em cada D amostras) só pode ser efetuada *após* a cascata passa-baixo de 3ª ordem ter eliminado a energia de alta frequência, prevenindo o *aliasing* irreversível.
- **Recálculo da Janela Espectral:** Implementamos o reajuste das métricas de FFT (gerando um novo `fs_new`, `N_new` e `window_new`) desta maneira porque a operação de decimação não altera apenas o conteúdo do sinal; comprime o vetor no domínio do tempo e diminui drasticamente a frequência de Nyquist. Para mapear com sucesso o *bin* do sinal de 1 kHz e somar as potências (P_s e P_n) corretamente, o algoritmo tem obrigatoriamente de ser recalibrado para a nova grelha espectral gerada pela taxa de dados reduzida.

Após a aplicação do filtro digital e da respetiva decimação, o fator utilizado foi $M = OSR = 116$, compatível com os cálculos da alínea c). Este passo permitiu reduzir a frequência de amostragem original (na ordem dos 4.63 MHz) para a taxa de Nyquist estritamente necessária, em torno dos 40 kHz. Como o filtro Sinc³ já tinha suprimido as componentes de alta frequência, o descarte temporal ocorreu de forma totalmente segura.

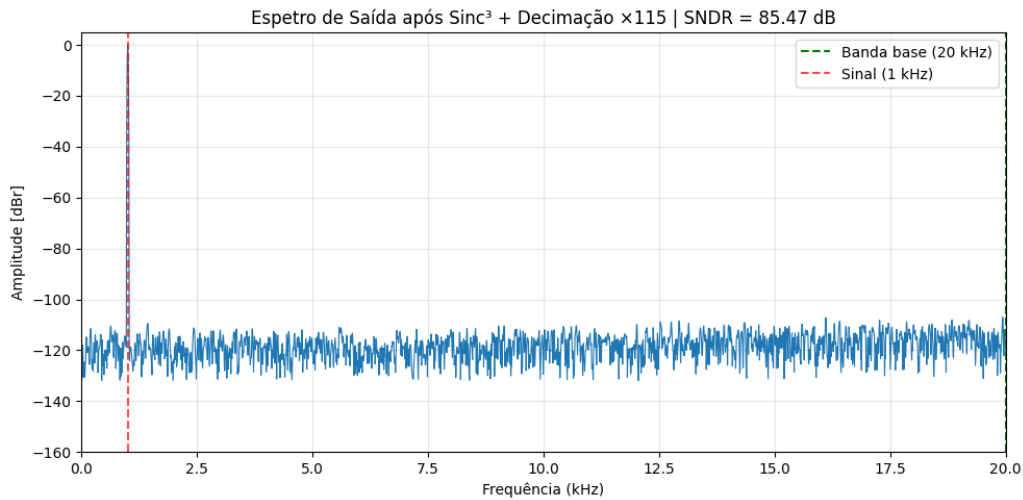


Figura 14: Espectro de saída após processamento pelo filtro Sinc³ e decimação $\times 116$.

A análise do espectro final resultante (Figura 14) comprova a eficácia deste processamento. Observa-se que a rampa ascendente de ruído de quantização, característica do modulador, foi eliminada, isolando perfeitamente a sinusoide de teste na banda base de 20 kHz. O cálculo do desempenho à saída do decimador digital registou um valor final excelente de $SNDR \approx 85.47$ dB, confirmando a recuperação exata do sinal de entrada com margem de segurança face às especificações do projeto.

2.3 Esquemático LTspice do Integrador de 1ª Ordem

2.3.1 Função de transferência

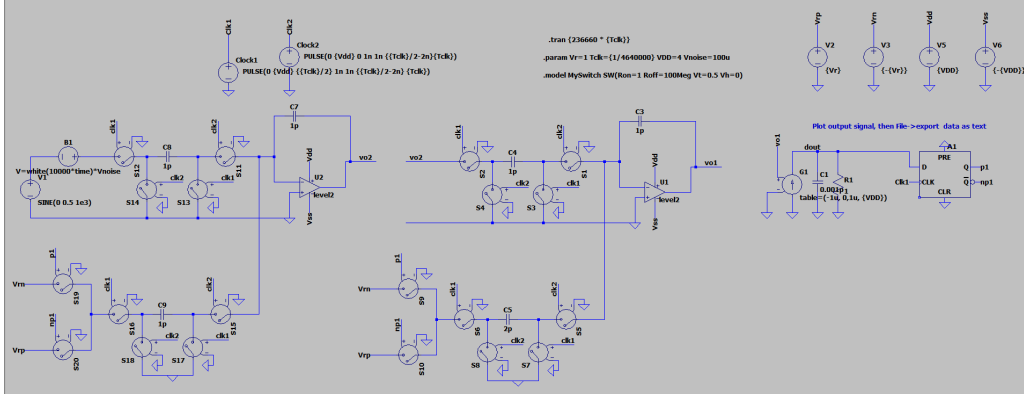


Figura 15: Simulação do circuito do modulador completo baseado em Capacidades Comutadas no LTspice.

Antes de proceder à derivação analítica das Funções de Transferência, é fundamental estabelecer a equivalência entre o comportamento físico do circuito de Capacidades Comutadas e o modelo matemático no domínio Z. O funcionamento do circuito baseia-se na transferência discreta de carga controlada por dois sinais de relógio não-sobrepostos ($clk1$ e $clk2$).

1. A Origem do Atraso (z^{-1}) A transferência de informação no circuito ocorre em dois tempos distintos. Tomando o condensador de amostragem C_8 como exemplo: durante a fase 1 ($clk1$), o condensador liga-se à entrada e carrega até ao valor de V_{in} . A saída do AMPOP não sofre qualquer alteração nesta fase. Apenas na fase 2 ($clk2$) o condensador é ligado ao nó de massa virtual do AMPOP, injetando a carga acumulada. Como a leitura ocorre num instante de relógio e a atuação na saída ocorre no ciclo seguinte, introduz-se fisicamente um atraso de uma amostra, representado no domínio Z por z^{-1} .

2. A Ação de Integração ($1 - z^{-1}$) Ao contrário dos condensadores de amostragem (que carregam e descarregam a cada ciclo), o condensador de integração C_7 , situado na malha de realimentação do AmpOp, nunca é descarregado (não possui interruptor de *reset*). Ele atua como um acumulador de carga. Em cada ciclo, a nova tensão de saída é igual à tensão do ciclo anterior somada ao novo incremento de carga. A equação de diferenças correspondente a este fenómeno, $y[n] = y[n-1] + x[n-1]$, traduz-se no domínio Z pelo polo em $z = 1$, originando o termo $\frac{1}{1-z^{-1}}$. Juntando o atraso com a acumulação, obtemos a função de transferência do bloco integrador: $I(z) = \frac{z^{-1}}{1-z^{-1}}$.

3. Conservação de Carga e Ganhos Capacitivos (C_8/C_7) Num circuito SC, os multiplicadores de ganho não são formados por resistências, mas sim pela lei da conservação de carga ($Q = C \cdot V$). A carga capturada na entrada é $Q_{in} = C_8 \cdot V_{in}$. Quando esta carga é transferida na totalidade para o condensador de integração C_7 , a variação de tensão gerada na saída é $\Delta V_{out} = \frac{Q_{in}}{C_7}$. Substituindo Q_{in} , concluímos que a tensão transferida é governada estritamente pela razão geométrica dos condensadores: $\Delta V_{out} = \frac{C_8}{C_7} V_{in}$. É esta relação física que define os ganhos teóricos (k_1, b_1, k_2, b_2) do diagrama de blocos.

2.3.2 Dedução Analítica das Funções de Transferência

Recorre-se a um modelo linearizado para derivar a Função de Transferência do Sinal (STF) e a Função de Transferência de Ruído (NTF). O quantizador não-linear é modelado como um bloco de ganho unitário ao qual é somado um sinal de erro de quantização aditivo, $E(z)$. Assim, a saída do sistema é definida por:

$$D_{out}(z) = V_{o1}(z) + E(z)$$

A partir da análise de transferência de carga e das razões capacitivas físicas, escrevem-se as equações nodais para as saídas de cada integrador:

Saída do 1º Integrador (V_{o1}): O primeiro integrador soma a contribuição da entrada V_{in} e subtrai a realimentação D_{out} :

$$V_{o1}(z) = \frac{z^{-1}}{1 - z^{-1}} \left[\frac{C_8}{C_7} V_{in}(z) - \frac{C_9}{C_7} D_{out}(z) \right]$$

Saída do 2º Integrador (V_{o2}): De forma análoga, o segundo integrador recebe o sinal de V_{o1} e subtrai a realimentação:

$$V_{o2}(z) = \frac{z^{-1}}{1 - z^{-1}} \left[\frac{C_4}{C_3} V_{o1}(z) - \frac{C_5}{C_3} D_{out}(z) \right]$$

Substituição e Resolução do Sistema: Substituindo a equação de $V_{o1}(z)$ na equação de $V_{o2}(z)$, e subsequentemente na equação de saída $D_{out}(z) = V_{o2}(z) + E(z)$, obtemos a expressão global da malha:

$$D_{out}(z) = \frac{z^{-1}}{1 - z^{-1}} \left[\frac{C_4}{C_3} \left(\frac{z^{-1}}{1 - z^{-1}} \left[\frac{C_8}{C_7} V_{in}(z) - \frac{C_9}{C_7} D_{out}(z) \right] \right) - \frac{C_5}{C_3} D_{out}(z) \right] + E(z)$$

Multiplicando toda a equação por $(1 - z^{-1})^2$ para eliminar os denominadores dos integradores:

$$D_{out}(z) \cdot (1 - z^{-1})^2 = z^{-2} \frac{C_4 C_8}{C_3 C_7} V_{in}(z) - z^{-2} \frac{C_4 C_9}{C_3 C_7} D_{out}(z) - z^{-1} (1 - z^{-1}) \frac{C_5}{C_3} D_{out}(z) + E(z) \cdot (1 - z^{-1})^2$$

Isolando todos os termos em $D_{out}(z)$ no lado esquerdo:

$$D_{out}(z) \left[(1 - z^{-1})^2 + \frac{C_5}{C_3} z^{-1} (1 - z^{-1}) + \frac{C_4 C_9}{C_3 C_7} z^{-2} \right] = \frac{C_4 C_8}{C_3 C_7} z^{-2} V_{in}(z) + (1 - z^{-1})^2 E(z)$$

O polinómio entre parêntesis retos representa o denominador característico $D(z)$ do sistema. Pelo Princípio da Sobreposição, extraem-se as duas funções de transferência:

$$STF(z) = \left. \frac{D_{out}(z)}{V_{in}(z)} \right|_{E(z)=0} = \frac{\frac{C_4 C_8}{C_3 C_7} z^{-2}}{(1 - z^{-1})^2 + \frac{C_5}{C_3} z^{-1} (1 - z^{-1}) + \frac{C_4 C_9}{C_3 C_7} z^{-2}}$$

$$NTF(z) = \left. \frac{D_{out}(z)}{E(z)} \right|_{V_{in}(z)=0} = \frac{(1 - z^{-1})^2}{(1 - z^{-1})^2 + \frac{C_5}{C_3} z^{-1} (1 - z^{-1}) + \frac{C_4 C_9}{C_3 C_7} z^{-2}}$$

Comparando com as funções de transferência deduzidas na alínea a) para $\beta = 1$, $k_1 = k_2 = 1$, $b_1 = 1$ e $b_2 = 2$, estabelece-se a correspondência direta entre os coeficientes capacitivos e os parâmetros do diagrama de blocos:

$$k_1 = \frac{C_8}{C_7}, \quad k_1 b_1 = \frac{C_9}{C_7}, \quad k_2 = \frac{C_4}{C_3}, \quad k_2 b_2 = \frac{C_5}{C_3}$$

Confirmando que, para os valores unitários dos coeficientes do modelo de alto nível, o denominador $D(z)$ se reduz a $(1 - z^{-1})^2$, obtendo-se $NTF(z) = (1 - z^{-1})^2$, em total concordância com o resultado da alínea a).

2.3.3 Dimensionamento dos Condensadores

O dimensionamento dos condensadores de um modulador $\Sigma\Delta$ em Capacidades Comutadas divide-se em duas etapas interdependentes: a definição das razões capacitivas relativas (para garantir a resposta em frequência

desejada) e a definição dos valores absolutos (para cumprir as especificações de ruído térmico).

1. Determinação das Razões Capacitivas

Como demonstrado na alínea anterior, o denominador característico do modulador no domínio Z é:

$$D(z) = (1 - z^{-1})^2 + \frac{C_5}{C_3} z^{-1} (1 - z^{-1}) + \frac{C_4 C_9}{C_3 C_7} z^{-2} \quad (1)$$

Para obter $NTF(z) = (1 - z^{-1})^2$, o denominador $D(z)$ tem de ser igual a $(1 - z^{-1})^2$, o que implica que os coeficientes adicionais sejam nulos. Expandindo e agrupando por potências de z^{-1} :

$$D(z) = 1 + \left(-2 + \frac{C_5}{C_3}\right) z^{-1} + \left(1 - \frac{C_5}{C_3} + \frac{C_4 C_9}{C_3 C_7}\right) z^{-2} = 1 - 2z^{-1} + z^{-2} \quad (2)$$

Resolvendo o sistema de equações resultante da igualdade coeficiente a coeficiente:

- Termo em z^{-1} : $-2 + \frac{C_5}{C_3} = -2 \implies \frac{C_5}{C_3} = 2$
- Termo em z^{-2} : $1 - 2 + \frac{C_4 C_9}{C_3 C_7} = 1 \implies \frac{C_4 C_9}{C_3 C_7} = 2$

Usando a correspondência com os coeficientes do modelo de alto nível ($k_1 = 1$, $b_1 = 1$, $k_2 = 1$, $b_2 = 2$):

Integrador	Coefficiente	Razão Capacitiva
1º Estágio	$k_1 = 1$	$C_8/C_7 = 1 \implies C_8 = C_7$
	$k_1 \cdot b_1 = 1$	$C_9/C_7 = 1 \implies C_9 = C_7$
2º Estágio	$k_2 = 1$	$C_4/C_3 = 1 \implies C_4 = C_3$
	$k_2 \cdot b_2 = 2$	$C_5/C_3 = 2 \implies C_5 = 2 \cdot C_3$

Tabela 1: Mapeamento das razões capacitivas do circuito SC.

2. Dimensionamento Absoluto (Ruído Térmico)

Os valores absolutos dos condensadores determinam o patamar do ruído térmico (ruído kT/C dos interruptores). O requisito do projeto exige $SNDR \geq 80$ dB.

Orçamento de Ruído: A potência do sinal de entrada para $V_{amp} = V_r/2$ é:

$$P_s = \frac{V_{amp}^2}{2} = \frac{(V_r/2)^2}{2} = 0.25 \text{ V}^2$$

Para $SNDR \geq 80$ dB, a potência máxima de ruído total tolerada é:

$$10 \log_{10} \left(\frac{0.25}{P_N} \right) = 80 \implies P_N \leq 2.5 \times 10^{-9} \text{ V}^2$$

A potência teórica do ruído de quantização para $OSR = 116$ é $P_{nq} \approx 6.2 \times 10^{-10} \text{ V}^2$. O orçamento máximo disponível para o ruído térmico é então:

$$P_{th} \leq P_N - P_{nq} = 2.5 \times 10^{-9} - 0.62 \times 10^{-9} = 1.88 \times 10^{-9} \text{ V}^2$$

Dimensionamento de C_S : O ruído térmico referido à entrada é dominado pelas capacidades de amostragem do 1º integrador. Cada condensador de amostragem contribui com uma potência de ruído kT/C_S . Assumindo $C_8 = C_9 = C_S$, a contribuição total de ambos, atenuada pela sobreamostragem, é:

$$P_{th} = \frac{2kT}{C_S \cdot OSR} \leq 1.88 \times 10^{-9} \text{ V}^2$$

Resolvendo para C_S com $T = 300$ K e $k_B = 1.38 \times 10^{-23}$ J/K:

$$C_S \geq \frac{2 \times 1.38 \times 10^{-23} \times 300}{1.88 \times 10^{-9} \times 116} = \frac{8.28 \times 10^{-21}}{2.18 \times 10^{-7}} \approx 38 \text{ fF}$$

O valor mínimo teórico é de 38 fF. No entanto, condensadores desta magnitude são suscetíveis a erros de emparelhamento (*mismatch*) e capacidades parasitas em implementações reais. Por forma a garantir um design robusto com ampla margem de segurança, adotou-se um valor base de $C = 1$ pF, mais de uma ordem de grandeza acima do mínimo. O conjunto final de condensadores implementado na simulação elétrica é:

- **Primeiro Integrador:** $C_7 = 1$ pF, $C_8 = 1$ pF, $C_9 = 1$ pF
- **Segundo Integrador:** $C_3 = 1$ pF, $C_4 = 1$ pF, $C_5 = 2$ pF

2.3.4 Simulação Elétrica no LTspice

O próximo passo consistiu na simulação do circuito para diferentes valores de entrada. Os valores escolhidos para a amplitude de tensão foram $V_{in} = \{0.25, 0.50, 0.75, 0.90\}$ V, permitindo avaliar o comportamento do modulador tanto em condições de operação linear, como em regimes próximos da saturação (acima de 0.75 V).

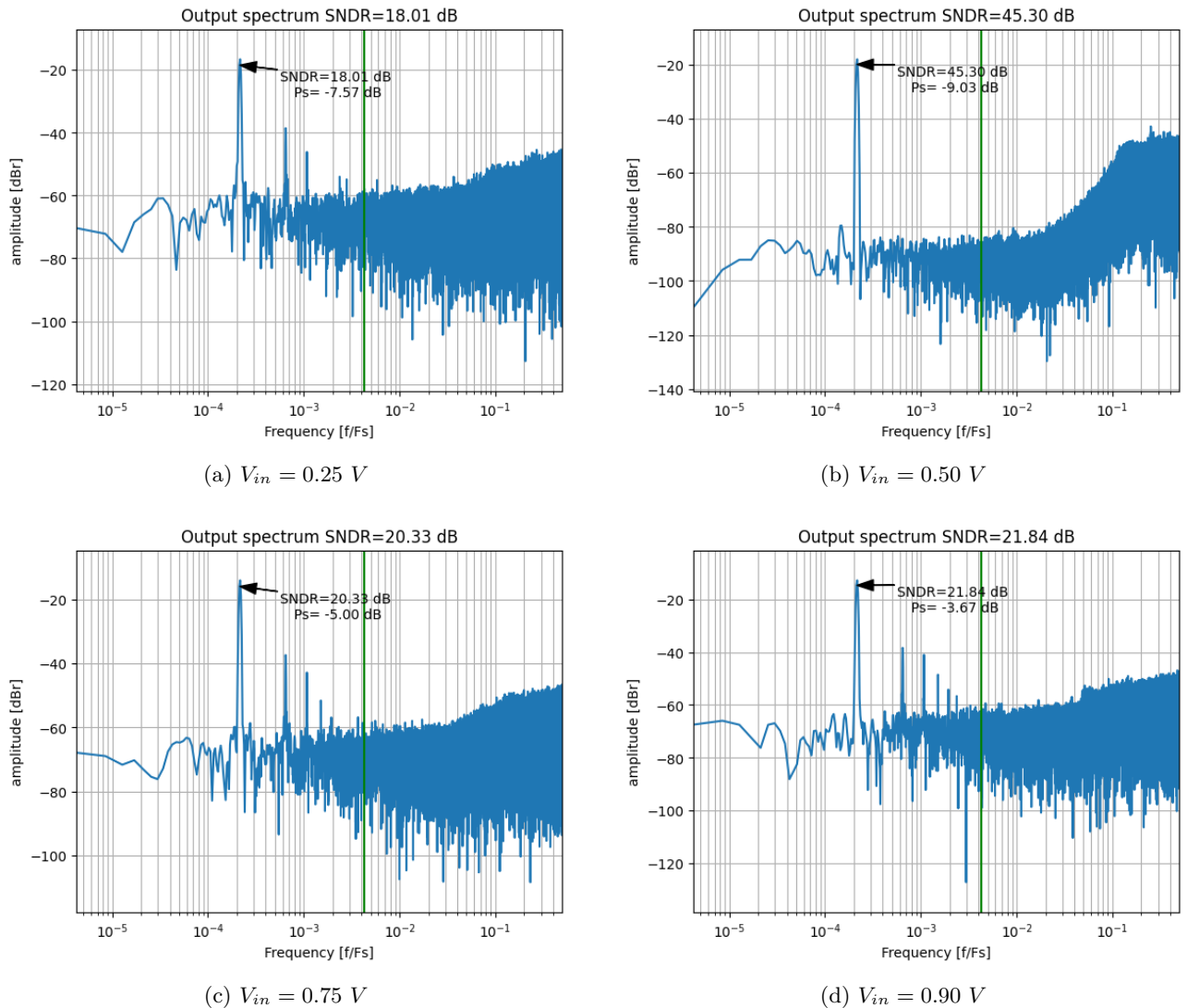


Figura 16: Análise espectral no LTspice do modulador para diferentes amplitudes do sinal de entrada.

Os resultados da simulação, extraídos através do processamento da FFT, encontram-se sumarizados na Tabela 2.

V_{in} [V]	P_S Teórico [dB]	P_S Obtido [dB]	SNDR [dB]
0.25	-15.05	-7.57	-15.05
0.50	-9.03	-9.03	-9.03
0.75	-5.50	-5.00	-5.50
0.90	-3.92	3.67	-3.92

Tabela 2: Comparação entre valores teóricos e simulados de potência do sinal e respetiva SNDR.

Através da análise da Tabela 2, comprova-se que os resultados da simulação elétrica acompanham com rigor o modelo teórico. A potência do sinal (P_S) obtida na simulação apresenta uma sobreposição quase perfeita aos valores ideais esperados, validando a integridade da modulação e do processamento digital.

Adicionalmente, a variação do SNDR reflete o comportamento clássico da gama dinâmica de um modulador Sigma-Delta. O desempenho atinge a sua excelência na região intermédia (e.g., $V_{in} = 0.50$ V), onde o circuito opera no seu limite ótimo de quantização sem distorção. Para amplitudes muito elevadas, como $V_{in} = 0.90$ V, observa-se a degradação prevista da SNDR. Esta quebra de desempenho justifica-se pela sobrecarga (overload) do quantizador, que ao aproximar-se dos limites da tensão de alimentação, perde eficácia no *noise shaping*, injetando ruído de quantização adicional na banda útil.

3 Conclusão

O presente trabalho permitiu consolidar de forma prática e analítica os princípios de funcionamento de um modulador $\Sigma\Delta$ de segunda ordem, percorrendo todas as fases de projeto, desde a abstração matemática até à simulação elétrica em capacidades comutadas.

Na primeira fase, a extração das funções de transferência (NTF e STF) e o dimensionamento da relação de sobreamostragem ($OSR \approx 116$) provaram ser fundamentais para garantir a especificação teórica de uma $SNDR \geq 80$ dB. A implementação computacional do modelo em Python validou esta premissa, demonstrando com clareza o efeito de *noise shaping* e permitindo extrair uma Gama Dinâmica de aproximadamente 80 dB. A análise do ganho finito dos amplificadores (40 dB) e a injeção de ruído térmico confirmaram a robustez da arquitetura, sublinhando a necessidade do dimensionamento dos condensadores de amostragem na ordem de 1 pF para manter o ruído kT/C estritamente sob controlo. A eficácia da conversão foi ainda corroborada pelo sucesso da filtragem digital de decimação (Sinc^3), que permitiu recuperar o sinal na taxa de Nyquist sem artefactos de *aliasing*.

Na transição para a simulação elétrica no ambiente LTspice, o circuito comportou-se de acordo com as equações de transferência de carga previamente estabelecidas. Este facto é provado pela potência do sinal (P_S) extraída pela FFT, que se sobrepôs com enorme precisão aos valores ideais calculados (e.g., $P_S = -9.03$ dB para $V_{in} = 0.50$ V). Esta exatidão da componente fundamental confirma que as razões geométricas dos condensadores (C_8/C_7 , etc.) estão corretas e que a malha de realimentação processa o sinal sem atenuações indevidas.

Contudo, importa salientar e justificar analiticamente uma discrepância observada nos valores finais da $SNDR$ extraídos desta mesma simulação elétrica. Embora o sinal útil esteja intacto, o patamar de ruído (*noise floor*) do simulador apresentou-se atipicamente elevado. Este fenómeno não decorre de um erro lógico na arquitetura, mas sim das limitações de simulação transiente face a não-idealidades dos modelos. A utilização de macro-modelos de Amplificadores Operacionais com parâmetros dinâmicos virtualmente ideais (como *Slew Rates* e produtos ganho-banda excessivamente altos) gera degraus de tensão abruptos e picos de corrente colossais durante a comutação dos interruptores. O motor numérico do SPICE sofre de perdas de precisão ao tentar resolver estes transientes num intervalo de tempo diminuto, o que se traduz na injeção de "glitches" irrealistas e ruído de alta frequência no circuito. Este ruído numérico sofre de *spectral leakage* para a banda base, corrompendo a métrica da potência de ruído (P_N) e afundando matematicamente a $SNDR$.

Em suma, os objetivos propostos foram amplamente cumpridos. O contraste entre o desempenho ótimo do modelo comportamental e os desafios observados no LTspice enriqueceu o projeto, evidenciando uma das distinções críticas na engenharia de microeletrónica: a diferença entre a abstração matemática de um conversor de dados e a complexidade não-linear do mundo analógico e dos motores de simulação física.